

2

The Gist of Claude Code

This chapter should help you get comfortable with Claude Code by walking through how it actually works in practice. Instead of jumping straight into advanced features, we start by building an understanding of the system itself, the terminology it uses, and the way it interacts with a code base. Along the way, you will see how Claude Code observes a project, builds context, and uses that context to assist with coding tasks.

We start by setting up a simple Next.js project and integrating it with Claude Code. This gives us a practical way to explore concepts like `CLAUDE.md` files, the permission system, and why being deliberate about context matters. We also look at how Claude Code can be extended using external tools through the **Model Context Protocol (MCP)**, and what that enables in day-to-day workflows.

The chapter also introduces spec-driven design as a way to guide Claude's behavior before any code is written. By creating and referencing a specification, we can influence the quality and consistency of the results we get back. As part of this process, we start building a project called HookHub. The HookHub project you build here will stay with us throughout the book and act as a shared reference point as we move into more advanced agentic workflows later on.

The following topics will be covered in this chapter:

- Setting up the Next.js project
- Connecting Playwright MCP to Claude Code
- Using spec-driven design
- Implementing the main page using spec-driven design

All right. Let's get to it!

NOTE

Installing Claude Code

Before proceeding, ensure that Claude Code is installed on your machine.

Claude Code now provides a native installer that detects your operating system and installs the appropriate version automatically. Follow the official installation guide:

<https://code.claude.com/docs/en/setup#native-install-recommended>

The installer handles setup and updates based on your machine type. Once installation is complete, you can verify it by running:

```
claude --version
```

After confirming Claude Code is installed correctly, continue with the Next.js setup below.

All terminal commands in this chapter are demonstrated on macOS. If you are using Windows, you should run these commands inside **WSL (Windows Subsystem for Linux)** or an equivalent Unix-like shell environment. This ensures that commands such as `npm`, `pnpm`, `claude`, and other CLI tools behave as shown in the examples.

Setting up the Next.js project

In this section, we will set up the Next.js project manually. Although Claude could have been prompted to create the Next.js project, I am doing this manually due to local permission issues and configurations that need to be handled beforehand. For those who prefer, Claude is fully capable of running these Bash commands when given the correct syntax.

We begin by setting up a Next.js project. You can follow the official Next.js installation guide (<https://nextjs.org/docs/app/getting-started/installation>) or paste the following command:

```
npx create-next-app@latest
```

As you can see from the following screenshot, we are in the **project/HookHub** branch.

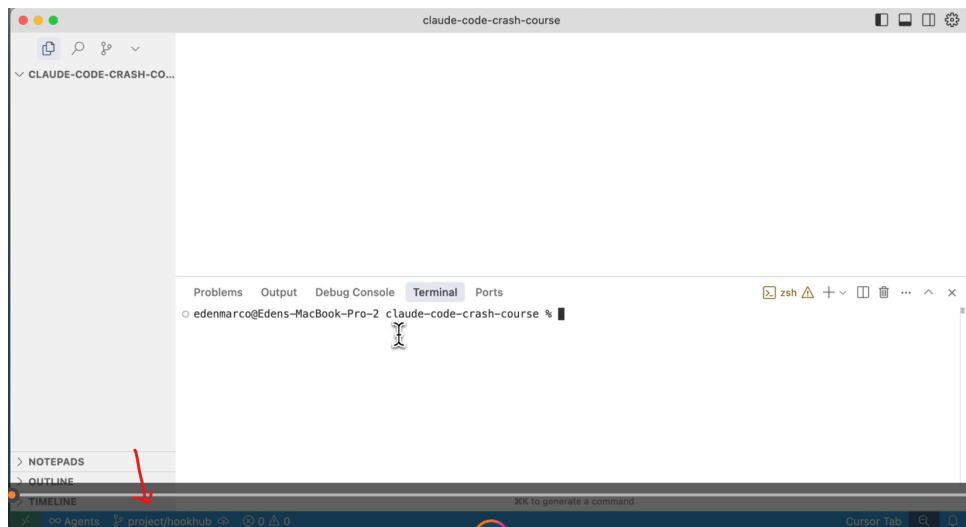


Figure 2.1 – project/hookhub branch

We paste the command and scaffold the Next.js project. You will get an option to proceed as shown:

```
edenmarco@Edens-MacBook-Pro-2 claude-code-crash-course % npx create-next-app@latest
Need to install the following packages:
create-next-app@15.4.6
Ok to proceed? (y)
```

After pasting the command, you may encounter an error. This occurs because the current user does not have the required permissions to install packages globally on the system.

There are two ways to resolve this. One option is to run the command with **sudo**. Another option is to adjust **npm** permissions so that the installation can proceed without elevated privileges. For simplicity, we rerun the command with **sudo**, which grants the necessary permissions.

Once executed with the correct permissions, the Next.js project scaffolding proceeds successfully. You will be prompted to confirm the configuration options. Select the default options and continue. The dependencies are then installed, and the boilerplate code for the Next.js project is generated.

After installation is completed, navigate into the newly created project directory:

```
cd hookhub
```

Now start the development server:

```
npm run dev
```

If you attempt to run the server from the wrong directory, the command will fail. Ensure that you are inside the project directory before running it.

Once the server starts successfully, open **http://localhost:3000** in your browser. You should see the default Next.js boilerplate application.

At this point, the Next.js project is running locally and ready to be integrated with Claude Code. After running the command again, we create the project hookhub. Select **Yes** for the prompts and install all the required dependencies. Once the setup is complete, you will be able to see the boilerplate code for the Next.js project.

Running and verifying the application

Once everything is set up, we navigate into the hookhub directory and run the project using **npm run dev**.

The Next.js application is now running. Open **localhost:3000** and you will see the standard Next.js boilerplate application as shown:

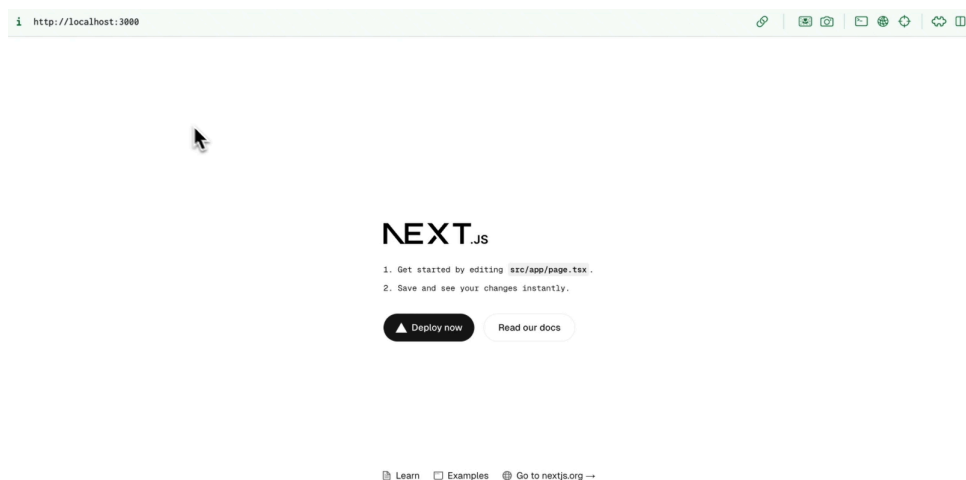


Figure 2.2 – Default Next.js application running on localhost:3000

Initializing Claude code

Before making further changes, return to Claude Code and provide it with the context of the project. This is done by initializing **CLAUDE.md** files so that they can be added to the project context.

Claude Code is opened through the Cursor integration. I personally prefer positioning Claude Code centrally and the code editor on the right, as shown. This setup allows Claude Code to act as the primary driver.

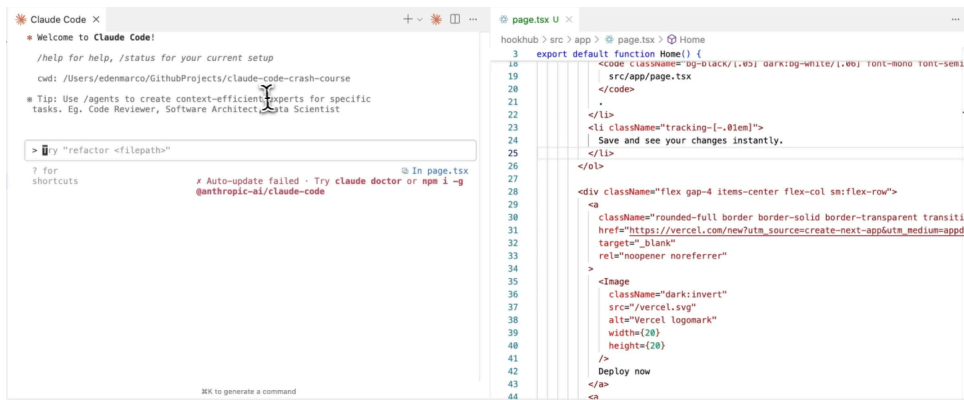


Figure 2.3 – Claude Code opened via Cursor Integration

The first step when initializing a project with Claude Code is to run the `/init` command. This applies whether the project is new or already contains many files. When `/init` is executed, Claude scans the files, checks their content, and begins generating reflections on the code base.

Claude identifies the tech stack, main content, and directory structure. All of this information is captured in a `CLAUDE.md` file, which will be explored later in the book. These Markdown files will eventually be distributed across the project to support context engineering. The purpose of these files is to send high-quality context to the LLM with every request. This makes it easier for the LLM to generate accurate and useful code.

While Claude runs, its status shows that it is **Booping**.

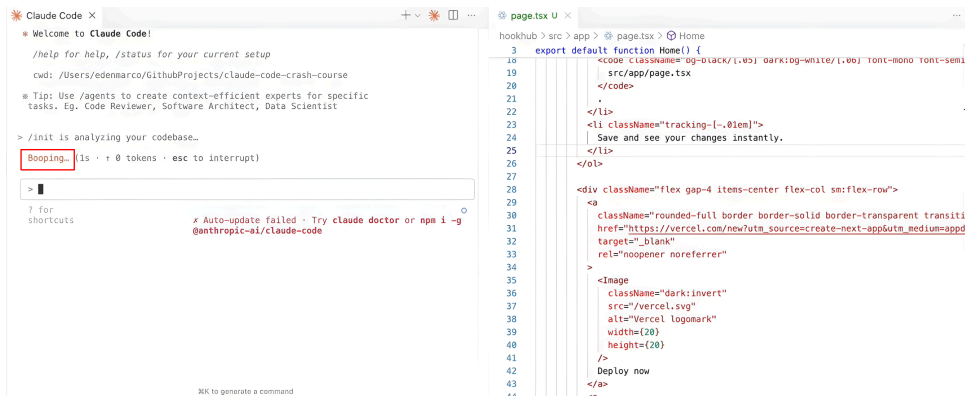


Figure 2.4 – Booping in Cursor Integration

In this context, Claude is analyzing the repository, exploring the directory structure, and reading file contents. After that, a task will be created to analyze the codebase structure, and a series of batch commands will be executed to list and read files.

Please keep in mind that this process can take longer for larger projects. Once it is done, Claude prepares to create the `CLAUDE.md` file based on the information it has gathered and will generate the following:

```
1 1+ # CLAUDE.md
2+
3+ This file provides guidance to Claude Code (claude.ai/code) when
4+ working with code in this repository.
5+ ## Project Overview
6+
7+ HookHub is a Next.js 15.4.6 application using the App Router
8+ architecture with TypeScript and Tailwind CSS.
9+ ## Essential Commands
10+
11+ ```bash
12+ # Development
13+ npm run dev          # Start development server on http://localhost:3000
14+
15+ # Production
16+ npm run build         # Create production build
17+ npm run start         # Start production server
18+
19+ # Code Quality
20+ npm run lint          # Run ESLint
21+ ```
22+
23+ ## Architecture
24+
25+ ...
```

Figure 2.5 – Generated CLAUDE.md file with project overview and commands

As you can see, a `CLAUDE.md` file has been created, containing the project structure and related details.

To save the file, Claude requests permission. We can either allow it to write the file and continue editing without further prompts or deny the change if we want to modify or discard it. This permission system applies to every new file or edit.

```
Opened changes in Cursor
Save file to continue...

Do you want to make this edit to CLAUDE.md?
1. Yes
2. Yes, and don't ask again this session (shift+tab)
3. No, and tell Claude what to do differently (esc)
```

This is an important part of practicing safe agentic coding. Allowing unrestricted access can quickly lead to unintended file changes and bugs. Maintaining control ensures safer workflows.

Select **Yes** for now and choose not to be asked again during this session. Claude completes the process and creates the `CLAUDE.md` file.

Contents of the CLAUDE.md file

The file includes the project architecture, structure, and basic information such as the following:

- The project goal
- The business problem being solved
- How the project is run
- The frameworks in use, including Next.js
- The folder and directory structure

From this point on, whenever prompts are sent to Claude Code, it first loads the relevant `CLAUDE.md` files. These files evolve dynamically as the project changes. When major updates

occur, the `/init` command can be run again, and additional `CLAUDE.md` files can be added as needed.

As more metadata and structural information are provided, Claude is able to behave more consistently and generate higher-quality results.

Claude Code now provides a dedicated installer, which handles setup and updates automatically. If issues occur, rerunning the installer typically resolves them.

After exiting the current session and restarting, a new command called `security review` becomes available. This command introduces a sub-agent designed to assist with security reviews and penetration testing against common vulnerabilities. It analyzes your codebase and flags common issues such as insecure patterns, misconfigurations, and vulnerabilities aligned with OWASP Top 10 risks.

Sub-agents will be covered in depth in [Chapter 3](#).

Connecting Playwright MCP to Claude Code

We begin by searching in Google for Playwright MCP. For those who do not know, Playwright is a browser automation tool that can be connected to Claude Code using the Model Context Protocol (MCP). If you do not know what MCP is, do not worry, it is covered in depth in the book. There is also an entire chapter dedicated only to MCP.

You can think of MCP as external capabilities that we can equip Claude Code with. In this repository, there are instructions for how to connect Claude Code to the Playwright MCP. Scroll and find the following command:

```
claude mcp add playwright npx @playwright/mcp@latest
```

Copy the command and paste it into the terminal. At this point, we are using the Claude Code **command-line interface (CLI)**.

```
edenmarco@Edens-MacBook-Pro-2 claude-code-crash-course % claude
* Welcome to Claude Code!
  /help for help, /status for your current setup
  cwd: /Users/edenmarco/GithubProjects/claude-code-crash-course

> /mcp
  (no content)

> /exit
  (no content)

> █

? for shortcuts
1 MCP server failed · /mcp for info
o IDE disconnected
edenmarco@Edens-MacBook-Pro-2 claude-code-crash-course % claude mcp add playwright npx @playwright/mcp@latest █

⌘K to generate a command
```

Figure 2.6 – Running Claude Code CLI in the terminal

In the terminal, we enter the command as shown in the following:

```
edenmarco@Edens-MacBook-Pro-2 claude-code-crash-course % claude mcp add playwright npx @playwright/
```

As you can see, we specified the MCP name as `playwright` and used `npx` `@playwright/mcp@latest`, which is where Claude Code pulls the repository and runs the code. The code being run is the Playwright MCP, and this is what gives Claude Code the extra capability of browser automation.

We also added the `-s user` flag. This ensures the MCP works for every instance of Claude Code we open because it is applied at the user scope. By doing this, we are taking advantage of Claude Code's hierarchy system, which will be covered in depth in later sections.

Verifying the MCP connection

Now, we open Claude again and enter `/mcp`. After pressing `Enter`, we should see all the MCP servers connected to Claude Code.

```
Manage MCP servers

1. context7 ✓ connected · Enter to view details
> 2. playwright ✓ connected · Enter to view details

MCP Config locations (by scope):
• User config (available in all your projects):
  • /Users/edenmarco/.claude.json
• Project config (shared via .mcp.json):
  • /Users/edenmarco/GithubProjects/claude-code-crash-course/.mcp.json (file does not exist)
• Local config (private to you in this project):
  • /Users/edenmarco/.claude.json [project: /Users/edenmarco/GithubProjects/claude-code-crash-course]

For help configuring MCP servers, see:
https://docs.anthropic.com/en/docs/claude-code/mcp
```

Figure 2.7 – Verifying MCP Server Connections in Claude Code

We can see the `playwright` MCP, and we can also see another MCP called `context7`, which was configured earlier. This is another useful MCP that will be discussed later.

Using Playwright MCP to pen a browser

To verify the Playwright MCP integration, we enter `open browser` in Claude Code and add `cn-n.com`. At this point, we encounter the permission system again, similar to the one shown:

```
Tool use

playwright – Navigate to a URL(url: "https://www.cnn.com") (MCP)
Navigate to a URL

Do you want to proceed?
> 1. Yes
2. Yes, and don't ask again for playwright – Navigate to a URL commands in /Users/edenmarco/GithubProjects/claude-code-crash-course
3. No, and tell Claude what to do differently (esc)
```

Figure 2.8 – Playwright MCP Permission Prompt to Open a Browser

Claude Code prompts us with a tool use request, asking whether it can run the Playwright tool to navigate to the URL `cnn.com`.

This is an external tool that has been connected to Claude Code, not a built-in tool. A built-in tool would be something like editing a file, creating a file, or deleting a file. As before, select **Yes**, and choose not to be asked again for this specific tool.

A browser opens, and we are now on cnn.com. Claude Code has used the Playwright MCP for browser automation and opened the site.

This becomes useful when we want to test a user interface or perform UI automation.

Using cursor rules as additional context

Next, open a browser and search for **cursor directory**, or navigate directly to <https://cursor.directory/>. The Cursor directory is a place where useful Cursor rules can be found. These rules allow additional context to be provided to Cursor through Markdown files, such as personas for the AI coding assistant—for example, making it behave like a senior developer specialized in a specific field.

These Cursor rules also work with Claude Code. To see this in practice, look for a Cursor rule for Next.js, select one, and copy it as shown.

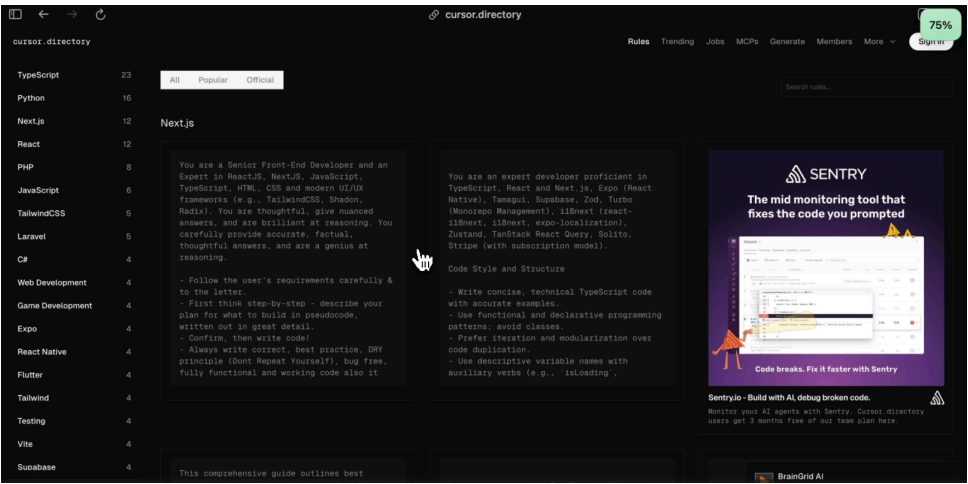


Figure 2.9 – Browsing Cursor Directory for Next.js Rules

Creating a frontend memory context

We return to Cursor and add this to a **CLAUDE.md** file. We already have the **CLAUDE.md** file that Claude created previously. We also have a broader memory system in which we can provide Claude with extra context.

In this case, we want to give it the Cursor rule and the persona of someone who knows Next.js very well, with the goal of producing better results. We first create a directory called **memory**, then create a subdirectory called **frontend**. Inside it, we create a file named **CLAUDE.md** and paste in the Cursor rule that you copied previously.

NOTE

The memory file mentioned will not be automatically loaded by Claude. This is intentional. Claude only auto-loads certain **CLAUDE.md** files by default. In this chapter, we are setting up the memory structure first. In [Chapter 3](#), we will configure when and how these memory files are loaded so they influence responses.

The rule that I copied contains content such as: *"You are a senior frontend developer and an expert in React, Next.js, JavaScript, TypeScript, HTML, CSS..."* and so on. In effect, we are giving Claude Code a more focused persona aligned with the tech stack we are using.

In the previous **CLAUDE.md** file, we told it that we are using a Next.js tech stack. In this **CLAUDE.md** file, we are giving it a persona.

At this point, it is clear that context is important for LLMs. So, we ask Claude the following question:

What are you expert in?

You might expect it to recite that it is an expert in frontend engineering, Next.js, TypeScript, and everything written in the `CLAUDE.md`. However, the response may be surprising.

The answer comes back as a generic statement:

```
> what r u expert in?

● I'm an expert AI assistant specialized in software engineering and coding tasks. I
  can help with:

  - Full-stack development - Frontend (React, Vue, Angular), Backend (Node.js,
    Python, Go, Rust, etc.)
  - Debugging and troubleshooting - Finding and fixing bugs, performance issues,
    type errors
  - Code refactoring - Improving code structure, readability, and maintainability
  - Building features - Implementing new functionality from scratch or extending
    existing code
  - Testing - Writing unit tests, integration tests, debugging test failures
  - DevOps - CI/CD, Docker, deployment configurations, build processes
  - Database work - SQL, NoSQL, migrations, query optimization
  - API development - REST, GraphQL, WebSockets
  - Security analysis - Defensive security, vulnerability detection, security best
    practices
  - Code reviews - Analyzing code quality, suggesting improvements
  - Documentation - When requested, creating technical docs and API documentation

  I have access to your file system and can read, write, and execute code directly.
  I also have web browsing capabilities and can fetch up-to-date library
  documentation through Context7.
```

Figure 2.10 – Generic Claude Response Without Loaded Project Context

As you can see, it says "expert AI assistant specialized in software engineering and coding tasks," without anything specific to Next.js or frontend. This makes it seem like the file is not working.

However, this is by design. Claude has a system of automatically loading `CLAUDE.md` files, and this will be covered in depth in [Chapter 3](#). There are also other files, called memory files, that are not automatically loaded.

Instead, we can explicitly tell Claude when to load them, depending on context. This opens the door to context engineering, which will be explored in the book.

The idea is that we can configure Claude to load the frontend context only when needed. For example, we can load it when designing or building the UI, when working only on TypeScript files, when working in specific directories, or when the request is clearly related to frontend work.

This gives us the freedom to customize Claude's behavior according to our needs. This will be demonstrated in the book, and one hint is that hooks can be used for this. We can reference context from other files, and there are several ways to work with memory, which will be covered in depth later in the book.

Using pec-driven design

In this section, instead of vibe coding the application, we use something called **spec-driven design**. This will be elaborated on later in the book, but the general idea is to create a spec file describing the application, the use case or business use case, and possibly some nonfunctional requirements.

The spec file that we create should tell Claude to reference the spec file when it is going to code. This helps optimize the context window and improves the results we get.

To begin, we press *Shift + Tab* and switch to **plan mode** in Claude Code. Plan mode is designed for tasks like this, where we want to do read-only work, conduct some research, and plan what Claude is going to do.

I have added the following prompt, in which I ask Claude to help write a spec file for a project called HookHub.

```
I want you to help me write a spec file for a project I am building. It's called "hookhub". It's a
```

As shown in the prompt, I have

- Described HookHub as a place where open-source Claude hooks are displayed and browsed
- Asked Claude to search for Claude hooks and write an initial specification
- Clarified that this is an MVP, with functionality limited to displaying hooks
- Specified that hooks are found in Git repositories and include a name, category, description, and a link to the repository
- Stated that the main page should display the hooks in a grid view

This is the full prompt. There is nothing fancy here, and no prompt engineering is applied.

Once we submit the prompt, Claude only uses read-only tools. No writing or editing occurs at this stage.

NOTE Since we are working in plan mode, we also need to select the model before pasting the prompt. Most of the time, when using plan mode, we prefer heavier models with deep reasoning capabilities. To control which model is used, we run the `/models` command and ensure that the latest version of Opus is selected. At the time of writing, this is the most advanced model available for deep reasoning. When the prompt is pasted, we only see a representation of the text rather than the actual text.

Claude begins by researching Claude hooks. It calls the web search tool with queries related to open-source Claude Code hooks and requests permission to fetch documentation from Anthropic, which we approve before proceeding.

After the research step is completed, Claude presents a plan similar to the following:

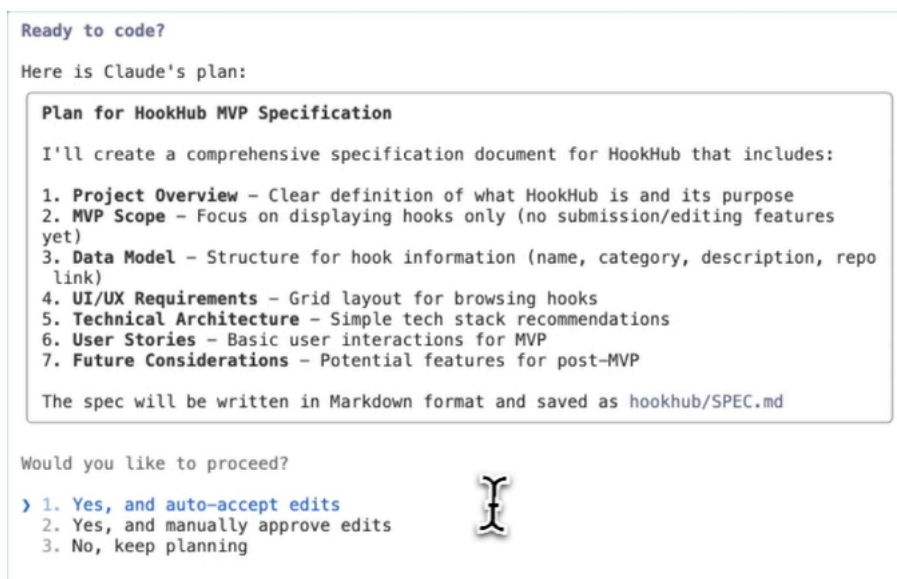


Figure 2.11 – Claude's Proposed Plan for HookHub MVP Specification

The plan includes the following:

- Creating a specification that covers the project overview
- MVP scope
- Data model for a Claude Code hook
- UI and UX requirements
- Technical architecture
- User stories
- Future considerations

At this point, we approve the plan. Claude then moves to write mode and requests to write the file to `hookhub/SPEC.md`. Decline and instruct Claude to write the spec under a `spec` directory and name the file `CLAUDE.md`.

I want you to write it into a `spec` directory and make this a `CLAUDE.md` file

Claude responds again and prepares to write the file under `spec/CLAUDE.md`. Approve this and allow the edit.

Claude will provide a to-do list for the first time. Claude begins iterating through the tasks and requests. We approve the actions as prompted.

Claude creates the `spec` directory and generates the `CLAUDE.md` file. We will go into much more detail about to-dos later in the book, but it is important to note that they play a major role in the quality of the results. This is one of the reasons Claude Code works well as a coding agent. Specifically, Claude Code behaves as a deep agent that can handle long-running, multi-step tasks that require significant reasoning. Deep agents and how Claude fits into this category will be covered later in the book.

Reviewing the generated spec

Once the process is completed, we now have a new `CLAUDE.md` file.

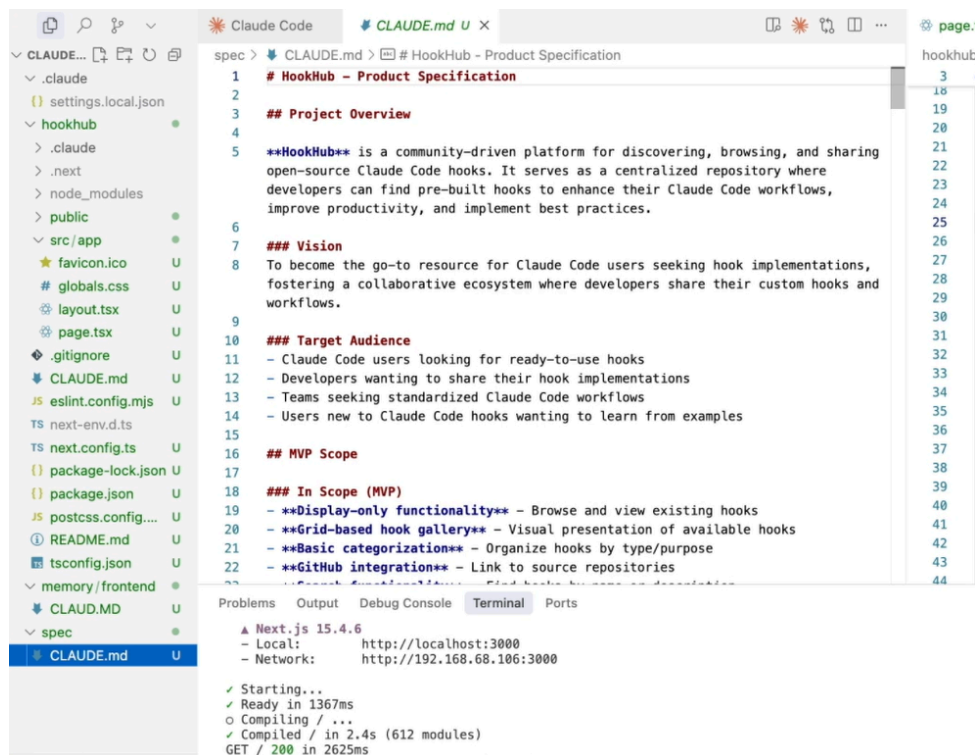


Figure 2.12 – HookHub Product Specification in `CLAUDE.md`

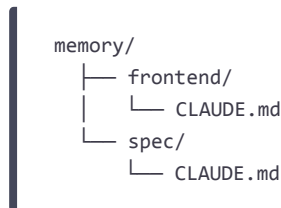
The file includes the project overview, vision, and a large amount of information that will help Claude during implementation.

The spec provides useful context, such as hook categories, hook object structure, and how the application should look. It also includes the steps required to build the project. This document will be added to the repository and will help Claude produce better results during implementation.

With this, you should have an idea of how using an LLM to help generate a spec is a common pattern. Other advanced patterns will be covered later in the book.

Finally, move the `spec` directory under the `memory` directory.

At this point, your structure should look like this:



This allows all long-term context to live in one place and be reused whenever it is needed.

Implementing the main page using the SPEC

Before we implement anything, we switch the model and use **Sonnet**. In the prompt, we write the following:

Can you please help me implement the main page grid as specified in the spec file?

We then use the `@` symbol to tag the spec file. The spec file we have is `spec/CLAUDE.md`. We will need to tag the file to attach it to the context.

NOTE

Every time we make a prompt, get a result, and then make another prompt, we keep filling the context. Over time, this results in many back-and-forth iterations between us and Claude.

If we continue using the same session without doing anything about it, we eventually fill the context window. When that happens, results start to get worse.

There are a couple of ways to handle this. There are custom commands, which will be covered later in the book, that help clear or summarize the context window while keeping the useful parts. For now, we use a simple, naive technique. Claude Code also provides built-in commands such as `/clear` and `/compact` for managing the context window without restarting the session. These were introduced earlier, and we will revisit them later. For now, restarting gives us a clean slate in the most straightforward way.

We exit the current Claude Code session by running `/exit`. After that, we open Claude again, navigate into the HookHub Next.js project, and open it from there. At this point, the context is fresh and empty.

We write the prompt again: *Can you please implement the main page, a grid of the hooks, as specified in the spec file?*

This time, we notice that the file reference cannot be found. The reason is that Claude Code was opened within the `HookHub` directory, but the `memory` directory is still at the root of the project.

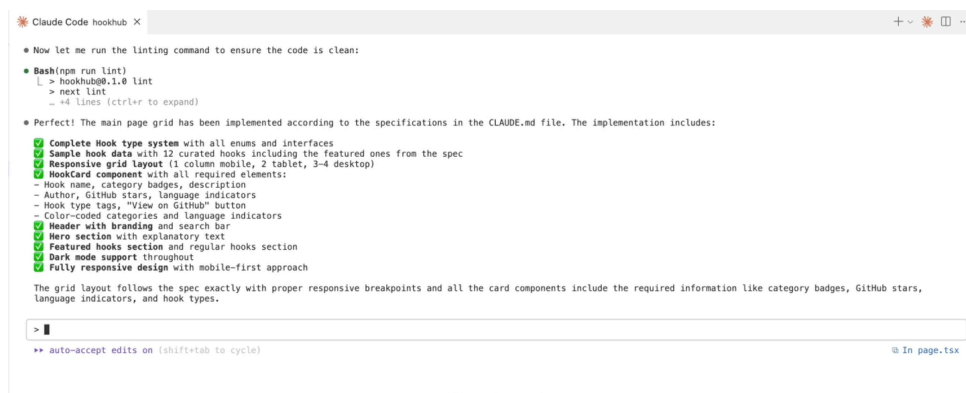
To fix this, we move the `memory` directory up in the hierarchy so that it sits under `hookhub`. Once this is done, the file reference works as expected. After this move, the `memory` directory is no longer at the crash course root level. It now lives inside the `HookHub` project directory.

Running the implementation

We write the prompt again to implement the grid according to the spec file. Claude now begins doing the heavy lifting. It creates a to-do list and starts implementing each task in the list.

A request will appear to approve file writes, such as creating the `Hooks.ts` file. Once approved, the implementation will continue and will take some time to complete.

Once the implementation is completed, the hook card component is created, and Claude continues running through the remaining steps. It then runs linting to catch potential errors. Once that finishes, we see a summary of what was implemented, including the card components, the hero section, the header, and other related pieces.



```

Claude Code hookhub x
+ v  [icons] ...

• Now let me run the linting command to ensure the code is clean:

• Bash(npm run lint)
  L > hookhub@0.1.0 lint
  > next lint
  - 4 lines (ctrl+e to expand)

• Perfect! The main page grid has been implemented according to the specifications in the CLAUDE.md file. The implementation includes:

  ✓ Complete Hook type system with all enums and interfaces
  ✓ Sample hook data with 12 curated hooks including the featured ones from the spec
  ✓ Responsive grid layout (1 column mobile, 2 tablet, 3-4 desktop)
  ✓ HookCard component with all required elements:
    - Hook name, category badges, description
    - Author, GitHub stars, language indicators
    - Hook type tags, "View on GitHub" button
    - Color-coded categories and language indicators
  ✓ Header with branding and search bar
  ✓ Hero section with explanatory text
  ✓ Featured hooks section and regular hooks section
  ✓ Dark mode support throughout
  ✓ Fully responsive design with mobile-first approach

  The grid layout follows the spec exactly with proper responsive breakpoints and all the card components include the required information like category badges, GitHub stars, language indicators, and hook types.

  > |
  >> auto-accept edits on (shift+tab to cycle) @ in page.tsx

```

Figure 2.13 – Claude Implementation Summary After Code Generation

Refresh the application in the browser and you will see a grid of Claude Code hook cards.

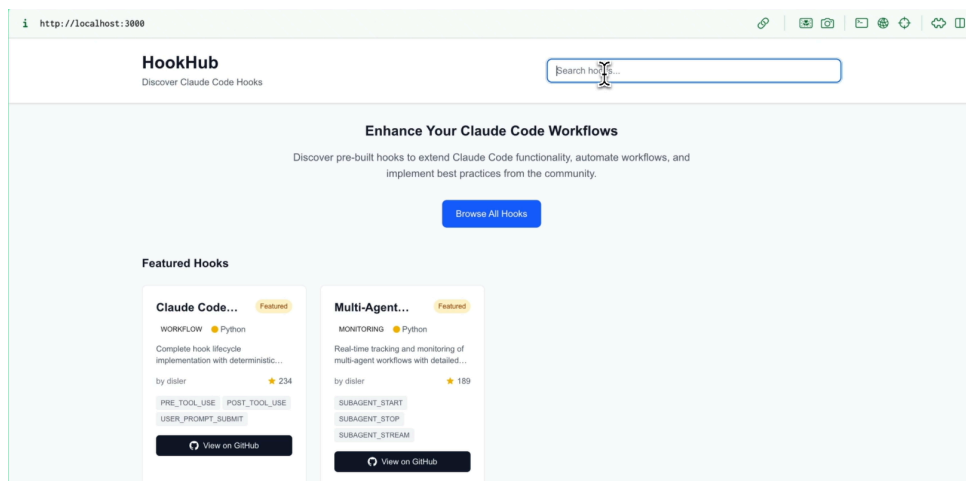


Figure 2.14 – Basic Next.js application

This is a basic Next.js application. While some features, such as search, are not yet implemented, others are already in place. Overall, it provides a solid skeleton that we can continue building on.

Later in the book, we will explore advanced agentic workflows for AI coding. This includes spinning up multiple sub-agents dynamically, iterating on UI components, and using meta-

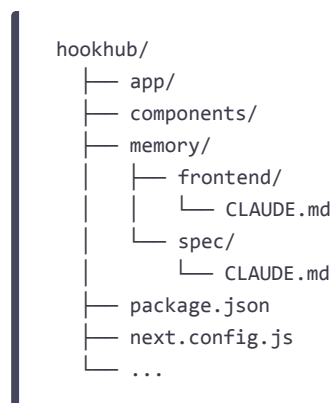
prompting. This project will serve as the foundation for those experiments.

At this point, we commit the code to the repository so it can be shared. We run `git status`, add all files, and confirm that there are no secrets or environment variables included.

We then commit the changes with a message such as *initial project* and push them to the repository.

From the repository, you can navigate to the `project/hookhub` branch, review the files, and pull the exact code from this commit. A command is also provided so you can clone the repository and get everything set up locally. To access the repository link, follow the steps in the *Download the example code files* section in the *Preface*.

After completing all the steps in this chapter, your HookHub project should have the following structure:



If your structure matches this, you are correctly set up for the next chapter.

Summary

In this chapter, we established a practical foundation for working with Claude Code by moving through each step of the setup and workflow. Rather than jumping straight into advanced features, we focused on understanding how Claude Code observes a project, builds context through `CLAUDE.md` files, and relies on explicit permissions to operate safely. This groundwork is essential because the quality of results you get from Claude Code depends directly on the quality of context, structure, and control you provide.

We also saw how Claude Code can be extended beyond its built-in capabilities using MCPs, how external tools like Playwright fit into the workflow, and why spec-driven design helps produce more consistent and predictable outcomes. The HookHub project created here is not just an example, it is a working baseline that will be reused as we explore more advanced agentic workflows later in the book. Every concept introduced in this chapter is something we will build on, rather than revisit from scratch.

In the next chapter, we will cover pricing and authentication, slash commands, user and project level configuration, and integration with Cursor.

Get this book's PDF copy, code bundle, and more

Scan the QR code (or go to packtpub.com/unlock). Search for this book by name, confirm the edition, and then follow the steps on the page.



UNLOCK NOW



invoice.